

3634. Minimum Removals to Balance Array

Solved 

Medium

 Topics

 Companies

 Hint

You are given an integer array `nums` and an integer `k`.

An array is considered **balanced** if the value of its **maximum** element is **at most** `k` times the **minimum** element.

You may remove **any** number of elements from `nums` without making it **empty**.

Return the **minimum** number of elements to remove so that the remaining array is balanced.

Note: An array of size 1 is considered balanced as its maximum and minimum are equal, and the condition always holds true.

Example 1:

Input: `nums = [2,1,5], k = 2`

Output: 1

Explanation:

- Remove `nums[2] = 5` to get `nums = [2, 1]`.
- Now `max = 2`, `min = 1` and `max <= min * k` as `2 <= 1 * 2`. Thus, the answer is 1.

Example 2:

Input: `nums = [1,6,2,9], k = 3`

Output: 2

Explanation:

- Remove `nums[0] = 1` and `nums[3] = 9` to get `nums = [6, 2]`.
- Now `max = 6`, `min = 2` and `max <= min * k` as `6 <= 2 * 3`. Thus, the answer is 2.

Example 3:

Input: `nums = [4,6], k = 2`

Output: 0

Explanation:

- Since `nums` is already balanced as $6 \leq 4 * 2$, no elements need to be removed.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^9$
- $1 \leq k \leq 10^5$

Python:

class Solution:

```
def minRemoval(self, nums: List[int], k: int) -> int:
    nums.sort()
    if nums[0]*k>=nums[-1]: return 0
    n=len(nums)
    ans=n
    l=0
    for r, x in enumerate(nums):
        while l<r and nums[l]*k<x:
            l+=1
        ans=min(ans, n-r+l-1)
    return ans
```

JavaScript:

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minRemoval = function(nums, k) {
    // Standard sort in JS sorts by string, so we need a comparator
    nums.sort((a, b) => a - b);
```

```

let i = 0;
let maxLen = 0;

for (let j = 0; j < nums.length; j++) {
  while (nums[j] > nums[i] * k) {
    i++;
  }
  maxLen = Math.max(maxLen, j - i + 1);
}

return nums.length - maxLen;
};

```

Java:

```

class Solution {
  public int minRemoval(int[] nums, int k) {
    Arrays.sort(nums);
    int i = 0;
    int maxLen = 0;

    for (int j = 0; j < nums.length; j++) {
      // Use long to prevent integer overflow for nums[i] * k
      while ((long) nums[j] > (long) nums[i] * k) {
        i++;
      }
      maxLen = Math.max(maxLen, j - i + 1);
    }

    return nums.length - maxLen;
  }
}

```